Fast Tamper Detection for Hierarchical Data in Live Cloud Forensics

Anyi Liu, Huirong Fu
Department of Computer Science and Engineering,
Oakland University
Rochester, Michigan 48309-4479
Email: anyiliu, fu@oakland.edu

***Abstract*—For digital forensics in the era of cloud computing, it is crucial that forensic artifacts are seized from a cloud infrastructure and analyzed in a timely manner. Unfortunately, when the forensic artifacts (e.g., the filesystem of the suspect's machine) are collected and transmitted from the cloud to the forensic investigator, in the form of a data stream, the metadata that describes the structure or hierarchy of the forensic artifacts might be lost. Even though researchers demonstrated the measures for detecting malicious tampering in the streaming data, an efficient approach is still desired to pinpoint the tampered location in a data hierarchy. In this paper, we present an efficient tamper detection scheme which is capable of pinpointing the location of the tampered data in the data hierarchy quickly, without the aid of metadata. Our scheme can be easily integrated into the existing streaming forensic data transmission schemes. The evaluation demonstrates both the effectiveness and the performance of the proposed scheme.**

I. Introduction

The proliferation of cloud computing provides the elastic applications and services with a much more cost-efficient solution. While the cloud computing offers the unique features, such as elasticity and low cost, it has also been extensively abused by the adversaries. For example, researchers show that the cloud-based exploits, malicious services, and ecosystem [1], [2], [3], [4] took advantages of the same appealing features offered by the cloud computing. As a consequence, there is an urgent need for the forensic investigators to respond to the reported criminal cases promptly, seize the data securely, and analyze the forensic artifacts (both physical and virtual) from the cloud timely.

As a guiding principle of digital forensic practice [5], it is crucial that "*No action taken by law enforcement agencies or their agents should change data held on a computer or storage media, which may subsequently be relied upon in court.*" Once the forensic artifacts are collected from the cloud infrastructure through the virtual machine introspection (VMI) [6], [7] and transmitted to the forensic investigator, both their confidentiality and integrity must be ensured and maintained in a chain-of-custody [8]. For a live cloud forensics process, it is even more important because the steps of a forensic process are pipelined. That is to say, the process of collecting the media, examining the data, analyzing the information, and reporting the evidences, should be performed in sequence, according to the definition of NIST [9]. Any security breach, which occurs in the the pipeline, will fail the whole forensic process. Although researchers have demonstrate how to facilitate live forensics in an IaaS, PaaS, and SaaS service models [10], some technical challenges still prevent live cloud forensics from being adopted in practice. First, when the excessive but volatile forensic artifacts are acquired from the cloud infrastructure, it should be sent to the forensic investigator immediately for analysis. If any part of the streaming data is tampered with during the transmission process, it is highly desirable that only the tampered portion to be resent, rather than the whole data set. Second, even if prior work demonstrates that the malicious tampering can be detected in a data stream [11], it is also important to accurately localize the tampered data. Unfortunately, when the forensic artifacts are transmitted as the streaming data, their semantic information and the meta-data are missing. For example, it is difficult to locate the tampered portion of the forensic artifacts in a hierarchy in nearly real-time. Of course, transmitting the metadata along with the raw data can solve the problem, but it suffers from at least two drawbacks. First, it will incur additional transmission overhead. Second, some pieces of information in the metadata might be redundant. For instance, given the metadata of "`/home`", "`/home/usr1`", and "`/home/usr2`", the substring "`/home`" is the common prefix for all the artifacts under the directory and thus will be transmitted with all the artifacts. Even the hash function can used to limit transmission overhead, in terms of the number of bits, the hash values cannot show any hieratical relationship between the artifacts either. For instance, given the metadata "`/home`" and "`/home/usr1`", their hash values are quite distinct. Therefore, it is difficult to use the hash values of "`/home`" and "`/home/usr1`" to identify their hieratical relationship within a directory.

In this paper, we describe an space-efficient scheme, which is particularly designed to identify the location of the tampered forensic artifacts, while they are transmitted between the cloud infrastructure and the forensic investigator in the form of streaming data. The unique advantage of this scheme is that it allows the location of the tamper data to be identified in a data stream as well as in a hierarchy. To do that, we design a novel algorithm that takes the advantage of the Cuckoo Filter (CF) [12] and a stack at the application level to store and retrieve the metadata. Once the tampered data is detected, the algorithm can quickly localize it and report its location in a hierarchy. The Cuckoo Filter is chosen due to its nice feature of reducing storage with an almost negligible false positive. Our theoretical analysis shows that the time complexity of

the new functionality of reporting the tampered location in a hierarchy is $O(1)$. Such a low overhead allows a forensic investigator to pinpoint the location of the tampered artifacts quickly. Our evaluation demonstrates both the effectiveness and the performance of the proposed approach are reasonable and hold great promise to be deployed in real-world applications.

The rest of the paper is organized as follows. Section II reviews the related work. Section III describes the threat model and the assumptions. Section IV presents system design and elaborate the methodology. Section V shows evaluation results. Finally, Section VI concludes the paper.

II. Related Work

Data Structures for Tamper Detection Several data structures have been used for the purpose of Tamper Detection. The non-binary Merkle Tree (MT) [13] can be used to verify the integrity of a hierarchical data set with a limited number of verification operations. However, to directly apply the MT in the tamper detection of streaming data is inappropriate. First, researchers demonstrate that the MT traversal requires logarithmic space and time complexity [14], which is not bounded. Second, since MT stores the hash of a data block in its leaf node and the hash of the child nodes in its non-leaf node, it lacks of the semantic information which can be used to identify the location in a hierarchy.

As a probabilistic data structure, the Bloom Filter (BF) [15] has been widely used in other applications, including the distributed storage system [16], Bitcoin [17], and digital forensics [18]. Unfortunately, one disadvantage of the BF is that it does not support the the delete operation. The Counting Bloom Filters (CBF) [19] and the Cuckoo Filters (CF) [12] are the two recent probabilistic data structures that achieve the goal of space-efficiency and support the delete operation. In our scheme, we choose the CF for at least two reasons. First, the operations of lookup and deletion in a CF is faster than that of a CBF. Second, the space complexity of the CF is better than that of the CBF because the space per bucket needs to be multiplied by the counter. Due to these advantages, we choose the CF over the CBF in our scheme.

III. The Threat Model and Assumptions

To specify our threat model without loss of generality, we assume that both the cloud provider and the forensic investigator have established mutual trust with each other through the mutual attestation protocols. More specifically, the mutual trust can be established by either measuring the trustworthiness of hardware, firmware, OS, and software stack of both parties involved in the data transmission through the commodity secure hardware [20], [21], [22], or construct the enclaves at the application-layer after the remote attestation [23]. In this paper, we do not consider the hardware attacks, such as TPM rollback attack [24], cold boot attack [25], memory disclosure attacks [26], CPU bus hijack attack [27], and system rebooting attack [28], which might compromise the physical security chips, OS, and hypervisor during the runtime. In addition, we also assume that once the mutual trust is established, the crypto-keys used in communication are *ephemeral* and thus cannot be stolen by the inside attacker through the race condition [29] and the side-channel attack [30]. In this paper, we particularly focus on the exploits that compromise data integrity in an insecure communication channel, which refers to the Lay 1 (network) according to the taxonomies defined by Dykstra et al. [31]. More specifically, our scheme defeats the following attacks:

- The *Man-in-the-middle (MITM) attack*, in which an adversary intercepts the transmitted data between the cloud provider and the forensic investigator and alter the data and its integrity digest.
- The *Force-to-retransmit attack*, in which an adversary deliberately corrupts a data fragment and cause a corrupted checksum of the whole dataset. If the tampered data cannot be precisely identified, the entire dataset might be forced to be retransmitted and thus waste the network bandwidth.
- The *Deniable attack*, in which either the malicious cloud provider or malicious forensic investigator will deny the reception of the data evidence.

Nevertheless, as a unique advantage, our approach can also identify the location of the tampered data in a data hierarchy.

IV. System Design

A. Definition

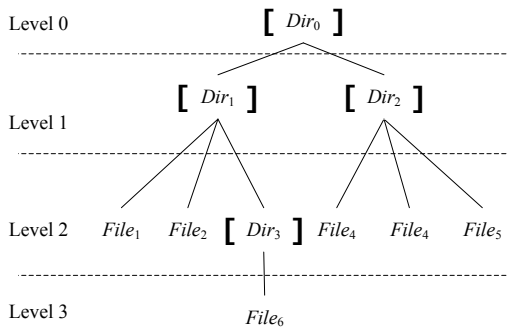
The key question that our scheme is intended to answer is how to quickly pinpoint the location of the tampered data in a hierarchy once it is detected from a data stream. To answer this question, it is required to define some terms that are used throughout this paper.

Definition 1. A *data stream* can be defined as $D = \{d_1, d_2, ..., d_n\}$, where d_i $(1 \leq i \leq n)$ is an indivisible *data element* in the stream. The data elements can be partitioned into m *data groups*, such that $D = \{G_1, G_2, ...G_m\}$ $(1 \leq l \leq m)$. A data group G_l can be clearly delimited if its last element d_o^l can be uniquely identified. We define the last element d_o^l of group G_l as the *boundary element* of G_l.

Definition 2. During the process of data collection in a cloud infrastructure, a *data hierarchy*, e.g., the file system, can be modeled as a tree T. A group of data units, e.g., a directory, can be modeled as a non-leaf node of T. A piece of data unit, e.g., a file, can be modeled as a leaf node of T.

Please note that a group of data units is different from the data group in Definition 1. Specifically, the former one refers to the data group exists in a data hierarchy, which contains the semantic information; while the latter one is the data group in a data stream, which is determined by the boundary element of the data group and thus does not contain any semantic information. In addition, since a data group is determined by its boundary element, which pertains to the specific boundary determination algorithm. The boundary element could be any arbitrary data element in a data stream.

Definition 3. Besides the *data groups*, one special group is defined as the *delimitation group* G_d, in which $G_d.value = Hash(Dir_i)$ $(1 \leq i \leq o)$ and $G_d.type \in \{$"[", "]"$\}$. Given a data hierarchy T, which is traversed by the *Iterative Deepening Depth-first Search* (IDDFS) algorithm, a G_d with the type of



The order of the hierarchy traversal without the delimitation group:
Dir_0, Dir_1, $File_1$, $File_2$, Dir_3, $File_6$, Dir_2, $File_3$, $File_4$, $File_5$

The order of the hierarchy traversal with the delimitation group:
[, [, $File_1$, $File_2$,], [, $File_6$,], [, $File_3$, $File_4$, $File5$,],]

Fig. 1: The order of a hierarch traversal.

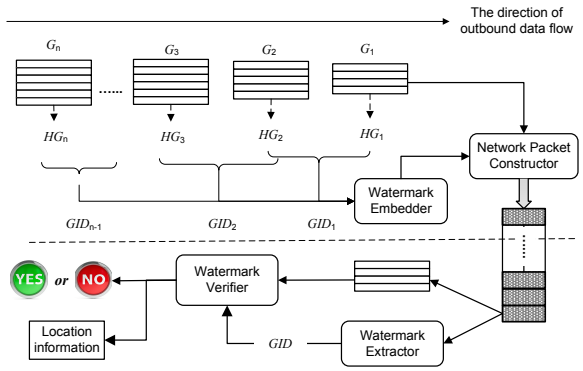


Fig. 2: The work flow of the proposed scheme.

"[". Likewise, a G_d with the type of "]"is inserted into the data stream after a non-leaf node has been traversed.

Figure 1 shows the delimitation groups and the occasions when they are inserted into the data stream. Given the depth of the traversal is 4, the order that the nodes of the tree are traversed is Dir_0, Dir_1, $File_1$, $File_2$, Dir_3, $File_6$, Dir_2, $File_3$, $File_4$, $File_5$. As mentioned previously, the delimitation group, whose type is "[" is inserted into the data stream before a non-leaf node is traversed. In the example, the non-leaf nodes are Dir_0, Dir_1, Dir_2, and Dir_3. Therefore, a delimitation group whose type is "[" inserted before them. Likewise, delimitation group whose type is "]" inserted after they and their sub-trees have been traversed.

B. The Workflow of the Scheme

Figure 2 illustrates the work flow of our prototype system, which implements the scheme and contains four major components: the *watermark embedder*, *watermark extractor*, *network packet constructor*, and *watermark verifier*. Its working mechanism is the following: The streaming data is first divided into the *data groups* G_i $(i = 1, 2, 3, ...n)$. Then, the group hash values of G_i are computed and denoted as $HG_i = H(G_i)(i = 1, 2, 3, ...n)$, in which the function H is a hash function. The two adjacent group hash values are concatenated and hashed again as $H(H(G_i)\|H(G_{i+1}))$, namely the *group integrity digest* (GID). Of course, the confidentiality of the GID can be guarded by the secret key or signed by the sender. The GIDs will be embedded into the network packets as the fragile watermark [32]. As previously mentioned, the appealing characteristic of the fragile watermark is that it is capable of tamper detection for malicious data modification, insertion, deletion without altering the data content and still remain transmission overhead negligible [33]. Once the network packets arrive on the receiver site, the watermark extractor first extracts the watermark from the packets. Meanwhile, the GIDs are computed from the raw data. If the extracted watermarks match the computed GIDs, then, the data integrity of that group is considered as passing the verification; otherwise, the data group(s) will be detected as the tampered group(s). The interested reader can refer to the algorithm for detecting the malicious modification, insertion, and deletion in [11], [32]. Nevertheless, as a unique contribution of the proposed scheme, our scheme can also tell the location of the tampered data in a hierarchical, along with the results of tamper detection.

C. Tamper Detection for Hierarchical Data

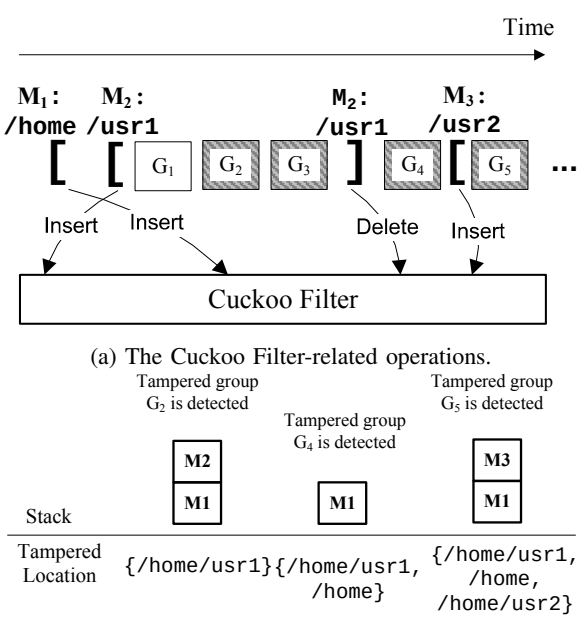
Once the delimitation group is inserted into the data stream, it is possible to tell the location of the tampered data in the hierarchy. Although the standard data structure, such as the hash table, the Merkle tree, and the blockchain specifically serve for that purpose, they suffer the same disadvantage that the spaces keep increasing while the number of unique keys are inserted into the data structure. For our problem, the number of the unique non-leaf nodes in an hierarchy is unknown upfront. Therefore, we are particularly interested in answering the question: *how to use a more space-efficient data structure to hold a set of unique non-leaf nodes, whose size could be arbitrarily large?*

To answer this question, we leverage the Cuckoo Filter, a probabilistic data structure with the features of high-speed, space-efficient, and supporting key deletion [12]. The Procedure *Identify_Locations* describes the algorithm that detects the locations of the tampered data in a hierarchy. It takes the data stream D, an un-initialized Cuckoo Filter CF, and an empty stack ST, as the input; and generates the output of the set of tampered locations STL, from which the tampered data is detected. Specifically, several subroutines are used in this algorithm: the subroutine *init* initializes the data structure of CF and ST. The subroutine *hasMoreGroup()* checks whether the streaming data contains more data groups. The subroutine *contains* checks whether CF contains a particular key. The subroutines *Insert* and *delete* are responsible for inserting the key and deleting the key from CF, repressively. The subroutines *push*, *pop*, *peek*, and *printStackTrace* follow the same definition as the standard ones for stack operations.

Procedure Identifying Locations of The Tampered Data - *Identify_Locations*

Input: The data group g_i in a data stream S, the Cuckoo Filter CF, the stack ST;
Output: The set of tampered locations STL, from which the tampered data were detected.

```
S ← ∅; init(CF); init(ST);
i ← 0;
while G_i ← S.hasMoreGroup() do
    i++;
    switch G_i.type do
        ▷ The starting delimitation group is identified
        case "["
            h ← DS(G_i);
            if CF.contains(D_i) == false then
                S.push(h);
                CF.Insert(h);
        ▷ The ending delimitation group is identified
        case "]"
            if CF.lookup(D_i) == true then
                h ← S.pop();
                CF.delete(h);
        ▷ The data group is identified
        case "Data"
            if detection(g_i) == true then
                Temp_D = peek(ST);
                if CF.contains(Temp_D) == true then
                    D_j = printStackTrace();
                    STL = STL ∪ {D_j};
End Procedure
```



(a) The Cuckoo Filter-related operations.

(b) The stack views and the set contained the tampered locations.

Fig. 3: The Cuckoo Filter and stack view of Example 1.

Example 1 We describe the working mechanism of the proposed algorithm through the example illustrated in Figure 3. In this example, the streaming data has already been grouped, which is denoted as G_1, G_2, G_3, G_4, G_5, and so on. The delimitation groups are M_1, M_2, and M_3, which are corresponding to the folders of "`/home`", "`/home/usr1`", and "`/home/usr2`", respectively. If the type of the delimitation group is "[", it will be inserted into CF. Meanwhile, the hash of that group is pushed into the stack ST. If the type of the delimitation group is "]", it will be deleted from CF and its hash will be popped out from ST. In this example, let us say that the groups G_2, G_3, G_4, G_5 are tampered in sequence and have been detected by the detection algorithm [11], the element at the top of the stack *ST* is first obtained, then is checked against *CF*. If *CF* contains the element, the stack track is print out and included into the set of locations TLS, from which the tampered data were detected. Therefore, when the tampered group G_2 is detected, M_2 will be obtained from the top of the stack. Since it has been inserted into CF before, the subroutine *printStackTrace* is performed and included the location "`/home/usr1`" into the *STL*. Similarly, the tamper group G_3 will include the same set into the *STL* and remains the *STL* unchanged. When tampered group G_4 is detected, the stack trace only contains one element M_1, and thus M_1 is the only one element in the stack and thus the location "`/home`" is included into the *STL*. Finally, the tampered group G_4 will identify that only M_1 and M_3 are in the stack. Therefore, the location "`/home/usr2`" is inserted into the *STL*. As the result, the location of the tampered group in a hierarchy is reported along with the detection of the data group in real-time.

The Analysis of Time Complexity The time complexity of the Procedure *Identify_Locations* is linear to the number of groups. For each group, the time complexities of inserting, deleting, and looking-up are all $O(1)$ [12]. In addition, the time complexities for the stack operations are also $O(1)$ [34]. Therefore, the accumulative time complexity of the algorithm is linear to the length of the data stream while the time complexity incurred by the Cuckoo Filter is almost negligible.

V. Evaluation

A. Tradeoff between Storage Overhead and False Positive Rate

This set of experiments first evaluates the false positive rate (FPR) of the Cuckoo Filter with the increasing number of the inserted metadata. Then, it compares the FPR of Cuckoo Filter with the Counting Bloom Filter. The parameters used in the evaluation are listed in Table I:

TABLE I: The parameters used by the Cuckoo Filter.

Symbol	Description	Value
α	the load factor	0.8
m	the number of buckets	4
b	number of entries per bucket	4
f	the fingerprint length in bits	20

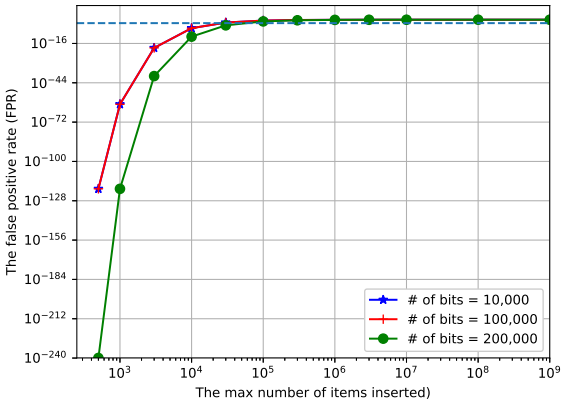
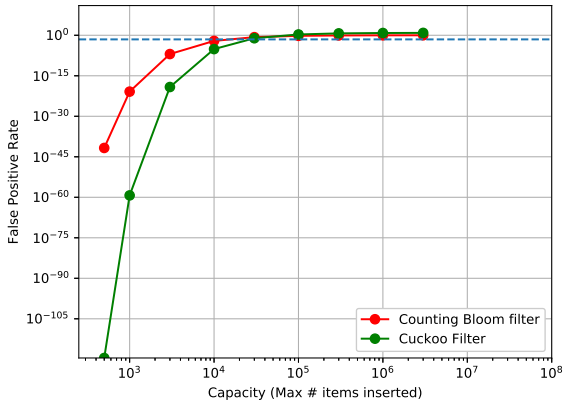
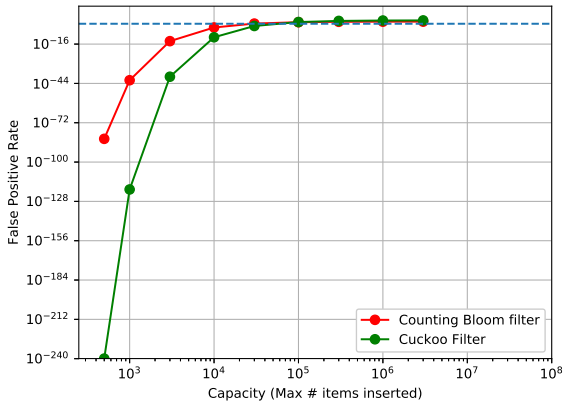


Fig. 4: The FPRs of CF with the different number of bits.



(a) NoB = 100,000.



(b) NoB = 200,000.

Fig. 5: The comparison of the FPRs between CF and CBF.

Figure 4 shows the FPRs of CF with the different number of bits. It is obvious that the increasing number of bits significantly reduce the FPRs of tamper detection, when the number of the inserted items is less than 10,000.

Figure 5a and Figure 5b illustrate the comparison of the FPRs between CF and CBF, when the number of bits (NoB) are 100,000 and 200,000, respectively. Compare the space efficiency between the usage of Cuckoo Filter and regular hash functions, our approach shows the efficiency of space. Given the same number of bits of storage, the proposed scheme can store more items than the regular hash functions with a considerably low FPR. As illustrated in Figure 5b, given a total number of 200,000 bits, our scheme can accommodate 10,000 distinct pieces of metadata with an FPR of 7.276e-12. However, it can only accommodate around 781 distinct pieces of metadata if the SHA-256 hash function is applied or 1562 distinct pieces of metadata if the MD5 hash function is applied. Therefore, the proposed scheme demonstrates much more space efficiency than the regular hash functions.

VI. Conclusions

To facilitate real-time tamper detection of data in a live forensic process, efficient algorithms are needed to not only detect the tampered data, but also its location in a data stream. In this paper, we present an efficient scheme, which is designed to detect the tamper of forensic artifacts, while they are transmitted in the form of streaming data. Our scheme can localize the tamper data in the streaming as well as pinpoint its location in a hierarchy. To accomplish these goals, a novel algorithm that combines the Cuckoo Filter (CF) and a stack is presented. The algorithm can quickly localize the tampered data and report its location in a hierarchy in real-time. Our theoretical analysis shows that the time complexity incurred by the new functionality of reporting tampered location in a hierarchy is $O(1)$, such that it allows forensic investigator to quickly pinpoint the location of the tampered artifacts. The evaluation demonstrated both the effectiveness and the performance of the proposed approach.

Acknowledgment

This work was supported in part by the National Science Foundation awards DGE-1723707, DGE-1623713, and the Michigan Space Grant Consortium. We also thank Pfarrer Michelle for the constructive discussion and the proofreading.

References

[1] S. T. King, P. M. Chen, Y.-M. Wang, C. Verbowski, H. J. Wang, and J. R. Lorch, "Subvirt: Implementing malware with virtual machines," in *Proc. 2006 IEEE Symp. Secur. and Privacy*, 2006, pp. 314–327.

[2] V. Ciancaglini, M. Balduzzi, R. McArdle, and M. Rösler, "Below the surface: Exploring the deep web," 2015, [Online]. Available: https://www.trendmicro.com/cloud-content/us/pdfs/security-intelligence/white-papers/wp˙below˙the˙surface.pdf.

[3] Symantec, "Avoiding the hidden costs of the cloud," [Online]. Available: https://www.symantec.com/content/en/us/about/media/pdfs/b-state-of-cloud-global-results-2013.en-us.pdf.

[4] R. Samani and F. Paget, "Cybercrime exposed: Cybercrime-as-a-servic," 2013, [Online]. Available: http://www.mcafee.com/jp/resources/white-papers/wp-cybercrime-exposed.pdf.

[5] Association of Chief Police Officers, "Good practice guide for computer-based electronic evidence - official release version 4.0," 2007.

[6] B. Hay and K. Nance, "Forensics examination of volatile system data using virtual introspection," *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 3, pp. 74–82, Apr. 2008.

[7] Y. Fu and Z. Lin, "Space traveling across vm: Automatically bridging the semantic gap in virtual machine introspection via online kernel data redirection," in *2012 IEEE Symposium on Security and Privacy*, May 2012, pp. 586–600.

[8] NIST Information Technology Laboratory, "NIST cloud computing forensic science challenges," 2013, [Online]. Available: http://csrc.nist.gov/publications/drafts/nistir-8006/draft˙nistir˙8006.pdf.

[9] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to integrating forensic techniques into incident response," [accessed June 10, 2015].

[10] S. Zawoad and R. Hasan, "Cloud forensics: A meta-study of challenges, approaches, and open problems," *CoRR*, vol. abs/1302.6312, 2013.

[11] A. Liu, J. Liu, and T. Uehara, "Secure streaming forensic data transmission for trusted cloud," in *Proc. 2nd International Workshop on Secur. and Forensics in Communication Systems*, June 2014, pp. 3–10.

[12] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than bloom," in *Proc. 10th ACM International on Conf. Emerging Netw. Experiments and Technologies*.

[13] R. C. Merkle, "A digital signature based on a conventional encryption function," in *A Conf. the Theory and App. of Cryptographic Techniques on Advances in Cryptology*, 1988, pp. 369–378.

[14] M. Szydlo, "Merkle tree traversal in log space and time," in *Advances in Cryptology - EUROCRYPT 2004*, C. Cachin and J. L. Camenisch, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 541–554.

[15] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970.

[16] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, "Bigtable: A distributed storage system for structured data," *ACM Trans. Comput. Syst.*, vol. 26, no. 2, pp. 4:1–4:26, Jun. 2008.

[17] A. Narayanan, J. Bonneau, E. Felten, A. Miller, and S. Goldfeder, *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*. Princeton, NJ, USA: Princeton University Press, 2016.

[18] S. Zawoad and R. Hasan, "I have the proof: Providing proofs of past data possession in cloud forensics," *CoRR*, vol. abs/1211.4328, pp. 75–82, 2012.

[19] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, "Summary cache: A scalable wide-area web cache sharing protocol," *IEEE/ACM Trans. Netw.*, vol. 8, no. 3, pp. 281–293, Jun. 2000.

[20] TCG, "D-RTM architecture specification version 1.0.0," Tech. Rep., [Online]. Available: https://trustedcomputinggroup.org/d-rtm-architecture-specification/.

[21] Intel, "Intel. 64 and IA-32 architectures software developer's manual," Tech. Rep., [Online]. Available: http://www.intel.com/content/www/us/en/processors/architectures-software-developer-manuals.html.

[22] Intel Corporation, "Intel trusted execution technology: Software development guide," Tech. Rep., [Online]. Available: http://download.intel.com/technology/security/downloads/315168.pdf.

[23] I. Anati, S. Gueron, S. Johnson, and V. Scarlata, "Innovative technology for CPU based attestation and sealing," *Workshop on Hardware and Architectural Support for Security and Privacy*, Jun. 2013, [Online]. Available: https://software.intel.com/en-us/articles/innovative-technology-for-cpu-based-attestation-and-sealing.

[24] B. Parno, J. R. Lorch, J. R. Douceur, J. Mickens, and J. M. McCune, "Memoir: Practical state continuity for protected modules," in *Proc. 2011 IEEE Symp. Secur. and Privacy*.

[25] J. A. Halderman, S. D. Schoen, N. Heninger, W. Clarkson, W. Paul, J. A. Calandrino, A. J. Feldman, J. Appelbaum, and E. W. Felten, "Lest we remember: Cold-boot attacks on encryption keys," *Comm. of the ACM*, vol. 52, no. 5, pp. 91–98, May 2009.

[26] L. Guan, J. Lin, B. Luo, J. Jing, and J. Wang, "Protecting private keys against memory disclosure attacks using hardware transactional memory," in *Proc. 2015 IEEE Symp. Secur. and Privacy*, 2015, pp. 3–19.

[27] J. Winter and K. Dietrich, "A hijackers guide to communication interfaces of the trusted platform module," *Comput. & Math. with App.*, vol. 65, no. 5, pp. 748–761, 2013.

[28] E. M. Chan, J. C. Carlyle, F. M. David, R. Farivar, and R. H. Campbell, "Bootjacker: Compromising computers using forced restarts," in *Proc. 15th ACM Conf. Comput. and Comm. Secur.*, 2008, pp. 555–564.

[29] D. Dean and A. J. Hu, "Fixing races for fun and profit: How to use access(2)," in *Proc. 13th USENIX Secur. Symp.*, 2004, pp. 14–14.

[30] M. Schwarz, S. Weiser, D. Gruss, C. Maurice, and S. Mangard, "Malware guard extension: Using SGX to conceal cache attacks," *CoRR*, vol. abs/1702.08719, 2017.

[31] J. Dykstra and A. T. Sherman, "Design and implementation of frost: Digital forensic tools for the OpenStack cloud computing platform," *Digit. Investig.*, vol. 10, pp. 87–95, Aug. 2013.

[32] H. Guo, Y. Li, and S. Jajodia, "Chaining watermarks for detecting malicious modifications to streaming data," *Information Sciences*, vol. 177, no. 1, pp. 281–298, 2007.

[33] A. Zigomitros, A. Papageorgiou, and C. Patsakis, "Social network content management through watermarking," in *Proc. 11th IEEE Int. Conf. Trust, Secur. Privacy Comput. Comm.*, 2012, pp. 1381–1386.

[34] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms, Third Edition*, 3rd ed. The MIT Press, 2009.